

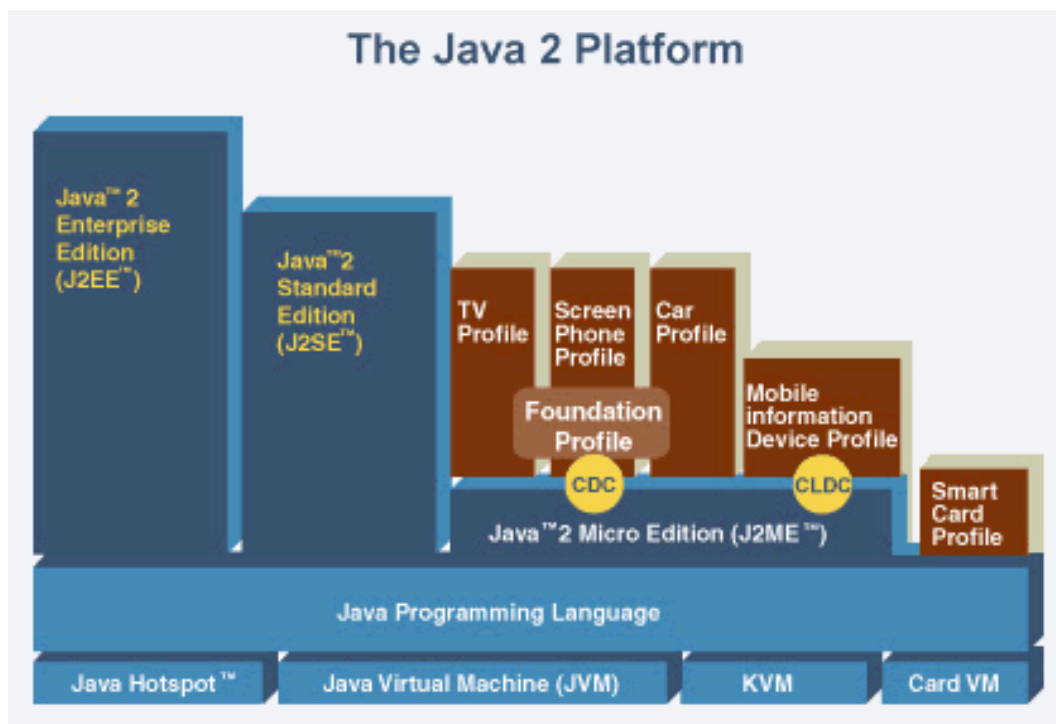
บทที่ 3

จาวาทูเอ็มอี

3.1 บทนำ

ภาษาจาวาเป็นแพลตฟอร์มการพัฒนาแอปพลิเคชันแบบเชิงวัตถุ ซึ่งจาวาทูเอ็มอีนี้เป็นจาวารุ่นหนึ่งของแพลตฟอร์มจาวา โดยแพลตฟอร์มจาวานั้นมีอยู่ 3 รุ่นด้วยกันดังนี้

- 1) J2EE (Enterprise Edition) เป้าหมายสำหรับการใช้งานในระบบงานใหญ่ ๆ เพื่อสามารถรองรับการใช้งานจาก ไคลเอนต์จำนวนมาก ๆ ได้
- 2) J2SE (Standard Edition) เป้าหมายสำหรับการใช้งานบนเครื่องคอมพิวเตอร์ตั้งโต๊ะทั่วไป ใช้ในการสร้างโปรแกรมประยุกต์แบบพึ่งพาตัวเองได้ หรือสร้าง แอ็พเพล็ต (Applet) โดยสมัยก่อนถ้าใครเคยใช้ JDK 1.1 ในการพัฒนาโปรแกรมก็คือตัวเดียวกันแต่เปลี่ยนชื่อมาเป็น J2SE ในจาวาทู
- 3) J2ME (Micro Edition) เป้าหมายสำหรับใช้งานบนอุปกรณ์ขนาดเล็ก โดยตัดฟังก์ชันที่ไม่จำเป็นออกไปเพื่อให้สามารถทำงานได้ในสภาวะที่มีทรัพยากรจำกัด



รูปที่ 3.1 แสดงแพลตฟอร์มของจาวาทู

จาวาทูเอมอีได้นำโครงสร้างแบบโมดูลที่มีความยืดหยุ่นสูงเข้ามาใช้งาน เพื่อให้สามารถรองรับการใช้งานที่หลากหลาย โดยชั้นของซอฟต์แวร์นั้นมี 3 เลเยอร์ด้วยกันโดยเลเยอร์ทั้งหมดจะอยู่เหนือระบบปฏิบัติการดังรูปที่ 3.1 ซึ่งมีดังนี้

เลเยอร์ที่ 1 : จาวาเวอร์ชวลแมชชีนเลเยอร์ (Java Virtual Machine Layer)

เป็นเลเยอร์ สำหรับการสร้าง จาวาเวอร์ชวลแมชชีนเลเยอร์ ซึ่งต้องปรับแต่งให้เข้ากับระบบปฏิบัติการของอุปกรณ์แต่ละชนิด และรองรับ การปรับแต่งแต่ละรูปแบบของจาวาทูเอมอี โดยจากรูปที่ 3.1 จะเห็นว่า เวอร์ชวลแมชชีน สำหรับ จาวาทูเอมอี ก็คือ CVM และ KVM

เลเยอร์ที่ 2 : เลเยอร์การปรับแต่ง(Configuration Layer)

การปรับแต่งจาวาทูเอมอีจะระบุตัว คลาสไลบรารี(class libraries) ตามกลุ่มของชนิดอุปกรณ์ที่ใช้งานทั่วไปภายใต้พื้นฐานความต้องการของ หน่วยความจำที่จำกัด และ หน่วยประมวลผลที่จำกัด จากรูปที่ 3.1 การปรับแต่ง(Configuration) ของ จาวาทู มี 2 แบบคือ CDC (Connected Device Configuration) และ CLDC (Connected Limited Device Configuration)

เลเยอร์ที่ 3 : โปรไฟล์เลเยอร์ (Profile Layer)

เป็นเลเยอร์ ที่ถูกสร้างไว้เหนือ เลเยอร์การปรับแต่ง โดยได้นิยาม คลาสไลบรารีต่าง ๆ เอาไว้เพื่อสำหรับ ความต้องการของตลาดเฉพาะกลุ่ม ดังรูปที่3.1 ตัวอย่างของ โปรไฟล์ในเลเยอร์ นี้ก็เช่น PDAP, MIDP, โปรไฟล์พื้นฐาน(Foundation Profile)และ โปรไฟล์บุคคล(Personal Profile) เป็นต้น

3.2 ส่วนการปรับแต่งใน จาวาทูเอมอี (Configuration)

การปรับแต่ง(Configuration) และ โปรไฟล์ ถือเป็นส่วนหลักสำคัญของ จาวาทูเอมอี ซึ่งจุดประสงค์ของการกำหนดสองส่วนนี้ก็เพื่อให้ได้ เวอร์ชวลแมชชีนและ คลาสไลบรารีที่เหมาะสมสำหรับอุปกรณ์แต่ละกลุ่มประเภท

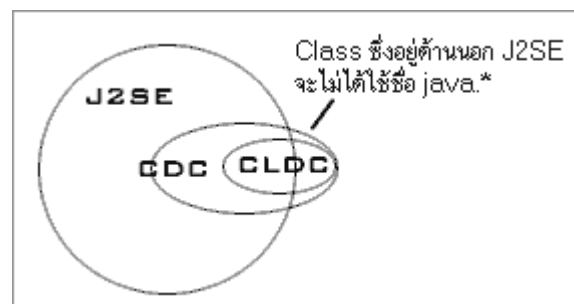
การปรับแต่ง(Configuration) หมายถึงชุดคุณสมบัติขั้นต่ำของ จาวาเวอร์ชวลแมชชีน และ จาวา คลาสไลบรารี สำหรับอุปกรณ์แต่ละประเภท หรืออาจกล่าวได้ว่า การปรับแต่ง(Configuration)นั้นเป็นตัวกำหนดคุณสมบัติขั้นต่ำของแพลตฟอร์มจาวาที่นักพัฒนาคาดว่าจะต้องมีในทุกอุปกรณ์ ขณะที่ คลาสไลบรารีที่กำหนดใน การปรับแต่ง(Configuration) จะมีในทุกอุปกรณ์ที่อยู่ในกลุ่มเดียวกัน

ปัจจุบันได้แบ่งการปรับแต่ง(Configuration) สำหรับจาวาทูเอมอี ออกเป็น 2 ประเภทคือ CDC (Connected Device Configuration) และ CLDC (Connected Limited Device Configuration) โดยมีเป้าหมายอยู่ที่อุปกรณ์ 2 ประเภทซึ่งมีความต้องการหน่วยความจำและหน่วยประมวลผลพื้นฐานใกล้เคียงกัน

CDC เป็นอุปกรณ์ใช้ร่วมกับอุปกรณ์อื่น ติดตั้งตายตัว และใช้เชื่อมต่อข้อมูล โดยปรกติแล้ว มักมีส่วนติดต่อกับผู้ใช้หลากหลายแบบมีหน่วยความจำประมาณ 2-16 เมกกะไบต์ ใช้หน่วยประมวลผลแบบ 32 บิตหรือมากกว่าเชื่อมต่อเครือข่ายที่มีช่องสัญญาณสูงโดยอาศัยพอร์ต TCP/IP ตัวอย่างอุปกรณ์ประเภทนี้ได้แก่ โทรศัพท์มือถือขนาดเล็ก อินเทอร์เน็ตทีวี โทรศัพท์พร้อมจอภาพที่เชื่อมต่อกับอินเทอร์เน็ต อุปกรณ์สื่อสารที่มีความซับซ้อนสูง อุปกรณ์บันทึกและระบบนำทางในรถยนต์

CLDC เป็นอุปกรณ์ส่วนบุคคล พกพาได้ และใช้เชื่อมต่อข้อมูล โดยปรกติจะมีส่วนติดต่อกับผู้ใช้แบบง่าย ๆ มีหน่วยความจำประมาณ 128 กิโลไบต์ ถึง 1 เมกกะไบต์ ใช้หน่วยประมวลผลแบบ 16 บิต หรือ 32 บิต เชื่อมต่อกับเครือข่ายที่มีแบนด์วิดท์ต่ำเป็นระยะเวลานาน ๆ โดยไม่อาศัยพอร์ต TCP/IP ตัวอย่างของอุปกรณ์พวกนี้ได้แก่ โทรศัพท์เคลื่อนที่ที่ไม่ซับซ้อนมาก เพจเจอร์รับส่งข้อความ เครื่องปาล์มโอเอสแบบพกพา

ในชั้นของ การปรับแต่ง(Configuration) นั้นมีคลาส 2 ประเภทด้วยกัน คือ คลาสที่นำมาจาก J2SE และที่ออกแบบเฉพาะอุปกรณ์ขนาดเล็ก คลาสที่นำมาจาก J2SE จะมีคุณสมบัติเดียวกับคลาสใน J2SE หรือเป็นซับคลาสใน J2SE เช่น แพคเกจ `java.lang` `java.io` และ `java.util`



รูปที่ 3.2 แสดงความสัมพันธ์ระหว่าง J2SE และคลาสไลบรารีใน CDC และ CLDC

3.3 เวอร์ชวลแมชชีนของจาวาทูเอ็มอี

เวอร์ชวลแมชชีน ของ CDC คือ ซีเวอร์ชวลแมชชีน(C Virtual Machine หรือ CVM) มีคุณสมบัติครบถ้วนเหมือน จาวาทูเวอร์ชวลแมชชีน แต่ขนาดเล็กกว่า ออกแบบมาสำหรับอุปกรณ์ที่สลับซับซ้อน CVM มีความ ต้องการหน่วยความจำ 256 กิโลไบต์ ขณะที่หน่วยความจำแบบ ROM ของ CDC มีขนาด 1 เมกกะไบต์

เวอร์ชวลแมชชีน ของ CLDC คือ เคเวอร์ชวลแมชชีน(K Virtual Machine หรือ KVM) ออกแบบมาใช้งานกับอุปกรณ์ที่มีทรัพยากรจำกัด ซึ่งมีหน่วยความจำประมาณ 128 กิโลไบต์ หน่วยประมวลผล แบบ 16 หรือ 32 บิต

3.4 โพรไฟล์ของจาวาทูเอ็มอี

โพรไฟล์ของจาวาทูเอ็มอีจะกำหนดชุดของ API (Application Programming Interface) ที่ต้องใช้เพิ่มเติม เช่น ส่วนติดต่อกราฟฟิกับผู้ใช้ การเชื่อมต่อเครือข่าย หน่วยเก็บข้อมูล เป็นต้น ซึ่งโพรไฟล์เหล่านี้ได้แก่

- โพรไฟล์พื้นฐาน (Foundation Profile) ซึ่งสร้างบน CDC โดยใช้งานกับอุปกรณ์ที่ต้องติดตั้ง จาวาเวอร์ชวลแมชีน ตัวเต็มควบคู่กับแพลตฟอร์มจาวาทู และ API ชุดมาตรฐาน
- RMI(Remote Method Invocation) โพรไฟล์ ซึ่งสร้างบน CDC โดยระบุชั้นชุดขั้นต่ำของ RMI ใน J2SE รุ่น 1.3
- โพรไฟล์บุคคล (Personal Profile) โพรไฟล์ ซึ่งสร้างบน CDC โดยมีเพื่อเป็นตัวเสริม เพอร์ซันนอลจาวา เอนไวรอนเมนต์(PersonalJava Environment) ของชั้นไมโครซิสเต็มส์
- PDAP (PDA Profile) ซึ่งสร้างบน CLDC มีส่วนติดต่อกับผู้ใช้ และ API ด้านการเก็บข้อมูล สำหรับอุปกรณ์พกพาขนาดเล็ก
- MIDP ซึ่งสร้างบน CLDC โดยเตรียมส่วนติดต่อกับผู้ใช้และหน่วยเก็บข้อมูล มีความสามารถด้านเครือข่าย แบบจำลอง API สำหรับโปรแกรมประยุกต์ไว้ให้อุปกรณ์ไร้สาย เช่น โทรศัพท์ที่ ซึ่งในโครงการนี้ได้ใช้ MIDP ในการทำบราวเซอร์บนอุปกรณ์ โทรศัพท์เคลื่อนที่

3.5 มิดเล็ต (MIDlet)

มิดเล็ต(MIDlet) หมายถึง โปรแกรมประยุกต์บน MIDP อาจกล่าวได้ว่า มิดเล็ตนั้นมีส่วนคล้ายกับจาวาแอปเพล็ต แม้จะไม่มีเมธอด main() แต่มิดเล็ตก็นำคลาส `javax.microedition.midlet.MIDlet` เข้ามาใช้ นอกจากนี้มิดเล็ตยังได้กำหนด คอนสตรักเตอร์ (constructor) แบบ สาธารณะ(public) ที่ไม่มีอาร์กิวเมนต์ใด ๆ อีกด้วย

วงจรการทำงานของ มิดเล็ต

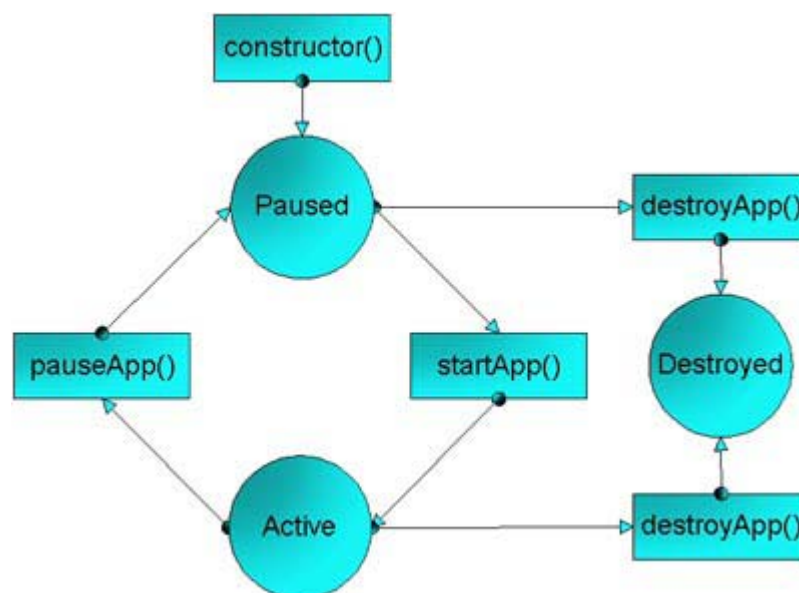
การทำงานของ มิดเล็ต ประกอบด้วย 3 สถานะคือ กำลังทำงาน หยุดทำงานชั่วคราวและถูกทำลาย การเปลี่ยนสถานะหนึ่งไปยังอีกสถานะหนึ่งควบคุมโดยเมธอด `startApp()` ,`pauseApp()` และ `destroy()`

เมื่อมิดเล็ต พร้อมสั่งกระทำการ ซอฟต์แวร์จัดการแอปพลิเคชันจะสร้างตัวอย่างมิดเล็ตขึ้นมา ก่อนโดยใช้ คอนสตรักเตอร์แบบสาธารณะ ที่ไม่มีอาร์กิวเมนต์ใด ๆ โดยมิดเล็ตจะอยู่ในสถานะหยุดชั่วคราว

จากนั้นซอฟต์แวร์จัดการแอปพลิเคชันจะเรียกเมธอด `startApp()` ขึ้นมา และ มิดเล็ตจะเข้าสู่สถานะ กำลังทำงานเปิดรับทรัพยากรที่ต้องการและเริ่มต้นการทำงาน ในสถานะนี้มิดเล็ตจะทำงานและดึง ทรัพยากรที่ต้องการไว้ใช้งาน เมื่อซอฟต์แวร์จัดการแอปพลิเคชันไม่ต้องการให้มิดเล็ตทำงานต่อไปก็

จะเรียกเมธอด `pauseApp()` จากนั้นมิดเล็ตจะหยุดทำงานและกลับเข้าสู่สถานะหยุดชั่วคราว คืนทรัพยากรที่ดึงมาใช้งานและเข้าสู่สถานะไม่ทำงาน มิดเล็ตจะสามารถเข้าไปสู่สถานะกำลังทำงานได้เมื่อซอฟต์แวร์จัดการแอปพลิเคชันเรียกเมธอด `startApp()` ขึ้นมา และเมื่อซอฟต์แวร์จัดการแอปพลิเคชันไม่ต้องการเรียกใช้งาน MIDlet หรือต้องการเคลียร์หน่วยความจำเพื่อให้โปรแกรมอื่นใช้งานได้ ก็จะส่งสัญญาณทำลายมิดเล็ตโดยการเรียกเมธอด `destroyApp()` แต่หาก มิดเล็ตอยู่ระหว่างการกระทำการขึ้นตอนสำคัญอยู่ ก็อาจร้องขอไม่ให้เข้าสู่สถานะถูกทำลายได้โดยเรียกใช้ `MIDletStateChangeException` อย่างไรก็ตาม ซอฟต์แวร์จัดการแอปพลิเคชันอาจปฏิเสธหรือยินยอมการร้องขอนี้ก็ได้ซึ่ง ตัวแปร `uncondition` ซึ่งเป็นตัวแปรบูลีนที่อยู่ใน `destroyApp()` นั้นหากเป็นจริง การร้องขอก็จะไม่ได้รับการสนอง แต่หากเป็นเท็จการร้องขอก็จะได้รับการตอบสนอง

หาก มิดเล็ต ต้องการหรือเข้าสู่สถานะหยุดชั่วคราวหรือถูกทำลายด้วยตัวเอง ก็สามารถทำได้โดยการเรียกใช้ เมธอด `notifyPause()`, `notifyDestroy()` ตามลำดับ



รูปที่ 3.3 แสดงวงจรการทำงานของ MIDlet

3.6 การใช้ API ระดับสูงในการพัฒนาส่วนติดต่อกับผู้ใช้

3.6.1 ลิสต์และข้อ

ลิสต์และข้อ กำหนดไว้ในแพ็คเกจ `javax.microedition.lcdui` โดยคลาสลิสต์นั้นมี คอนสตรักเตอร์ที่ถูกกำหนดไว้ 2 แบบดังนี้ คือ `List(String title,int listType)` และ `List(String title,int listType,String[] stringElements,Image[] imageElements)` โดย คอนสตรักเตอร์ ตัวแรกจะทำหน้าที่ในการสร้าง ลิสต์เปล่า ๆ ขึ้นมาซึ่งสามารถเพิ่มรายการของ ข้อเข้าไปได้ภายหลัง ส่วน คอนสตรักเตอร์ ตัวที่สองนั้นจะสร้างลิสต์ที่มีค่าเริ่มต้นอยู่ใน อาร์เรย์ `stringElements` ซึ่งต้องไม่เป็น `null` และสา

มาชิกทุกตัวในอาร์เรย์ต้องไม่เป็น null เช่นเดียวกันความยาวของอาร์เรย์ stringElements จะกำหนดรายการในลิสต์ อาร์เรย์ imageElements นั้นอาจมีค่าเป็น null ซึ่งหมายถึงรายการในลิสต์นั้นไม่มีภาพอยู่ ถ้าอาร์เรย์ imageElements นั้นไม่มีค่าเป็น null ก็จะต้องมีความยาวเท่ากับอาร์เรย์ stringElements รายการแต่ละรายการในอาร์เรย์ imageElements อาจจะเป็น null ได้หมายความว่าไม่มีภาพในรายการของลิสต์ที่ตรงกัน เมธอดทั้งหมดที่ใช้เรียกดูและเปลี่ยนแปลงรายการในลิสต์จะกำหนดไว้ในอินเตอร์เฟซข้อ

อินเตอร์เฟซ ข้อ จะกำหนด API สำหรับคอมโพเนนต์ส่วนติดต่อกับผู้ใช้โดยนำตัวเลือกจากรายการที่กำหนดไว้ล่วงหน้าจำนวนหนึ่งมาใช้ โดยอินเตอร์เฟซข้อ มีเมธอด เช่น

```
Int append(String stringPart,Image imagePart)
void delete(int elementNum)
Image getImage(int elementNum)
Int getSelectedIndex()
String getString(int elementNum)
void insert(int elementNum,String stringPart,Image imagePart)
boolean isSelected(int elementNum)
void set(int elementNum,String stringPart,Image imagePart)
void setSelectedIndex(int elementNum,Boolean selected)
int size()
```

ออบเจกต์ข้อมี 3 ประเภทได้แก่ อิมพลิชิต-ข้อ(implicit-choice),เอ็กคลูซิฟ-ข้อ(exclusive-choice),มัลติเปิล-ข้อ(multiple-choice)

- เอ็กคลูซิฟ-ข้อ:จะต้องเลือกรายการใดรายการหนึ่งในแต่ละครั้ง เว้นเสียแต่ว่าไม่มีรายการใด ๆ ให้เลือก
- อิมพลิชิต-ข้อ:เป็นกรณีพิเศษของ เอ็กคลูซิฟ-ข้อ เมื่อรายการที่อยู่ในโฟกัสถูกเลือกโดยอัตโนมัติเมื่อเริ่มต้นซึ่งข้อประเภทนี้ใช้กับลิสต์เท่านั้น
- มัลติเปิล-ข้อ:สามารถเลือกได้หลายรายการพร้อมกันในแต่ละครั้ง หรือจะไม่เลือกเลยก็ได้

การแก้ไขออบเจกต์ข้อ

เราสามารถแก้ไขเมธอด append(String stringPart,Image imagePart) และ insert(int elementNum,String stringPart) ในการเพิ่มออบเจกต์ข้อ ซึ่งหลังจากที่ได้ทำการเพิ่มแล้วเรายังสามารถแก้ไขภาพหรือสตริงได้โดยใช้เมธอดset() การลบออกเราก็ทำได้โดยใช้เมธอด delete(int elementNum)

3.6.2 เท็กซ์บ็อก (Text box)

คลาสเท็กซ์บ็อก เป็นรูปแบบที่มีโครงสร้างกำหนดไว้อยู่แล้ว ซึ่งผู้ใช้สามารถป้อนหรือแก้ไขข้อความในเท็กซ์บ็อกได้ การสร้างออบเจกต์เท็กซ์บ็อกนั้นใช้คอนสตรัคเตอร์ดังนี้

TextBox(String title,String text,int maxSize,int constraints) โดยฟังก์ชันที่ใช้ได้สำหรับเท็กซ์บ็อกมีดังนี้

```
void delete(int offset,int lenght)
int getCaretPosition()
int getChars(char[] data)
int getConstraints()
int getMaxSize()
String getString()
void insert(char[] data,int offset,int length,int position)
void insert(String src,int position)
void setChars(char[] data,int offset,int lenght)
void setConstraints(int constraints)
int setMaxSize(int maxSize)
void setString(String text)
int size()
```

ขนาดสูงสุด

ของข้อความในเท็กซ์บ็อก เนื่องจากว่า เท็กซ์บ็อกจะเก็บข้อความเป็นอาร์เรย์ของตัวอักษร ดังนั้นจึงมีขนาดจำกัด โดยเราสามารถกำหนดขนาดสูงสุดได้โดยใช้เมธอดดังนี้ TextBox(String title,String text,int MaxSize,int constrains) หรืออาจใช้เมธอด getMaxSize() ก็ได้ ซึ่งขีดจำกัดว่าเราสามารถสร้างเท็กซ์บ็อกให้มีขนาดเท่าไรนั้นอยู่ที่ MIDP เพราะว่าหน่วยความจำมีขนาดจำกัด

3.6.3 อเลิร์ท (Alert)

คลาส อเลิร์ท มีรูปแบบหรือโครงสร้างใช้สำหรับแสดงข้อมูลให้ผู้ใช้และจะคอยครุ่นหนึ่ง (กำหนดโดย timeout) ก่อนที่จะแสดงหน้าจอถัดไป คอนสตรัคเตอร์มี 2 เมธอดดังนี้

```
Alert(String title)
Alert(String title,String alertText,Image alertImage,AlertType alertType)
```


DataField(String label,int mode)

DataField(String label,int mode,TimeZone timeZone)

การป้อนข้อมูลสามารถทำได้ 3 โหมดด้วยกัน

- วัน(Date) -> ผู้ใช้กำหนดได้เฉพาะข้อมูลวันเดือนปีเท่านั้น
- เวลา(Time) -> ผู้ใช้กำหนดได้เฉพาะข้อมูลเวลาเท่านั้น ส่วนวัน เดือน ปี จะถูกกำหนดเป็น 1 มกราคม ค.ศ. 1970 และไม่สามารถแก้ไขได้
- วันเวลา(Date_Time) -> ผู้ใช้กำหนดได้ทั้งวันเดือนปีและเวลา

3.6.6 เท็กซ์ฟิลด์ (TextField)

คลาส เท็กซ์ฟิลด์ เป็นไอเท็ม ที่มีสตริงข้อความอยู่ซึ่งให้ผู้ใช้สามารถแก้ไขข้อความได้ การสร้างออบเจ็กต์เท็กซ์ฟิลด์ สามารถทำได้โดยใช้เมธอด TextField(String label,String text,int maxSize,int constraints)

เท็กซ์ฟิลด์ จะมีรูปแบบคล้าย ๆ กับเท็กซ์บ็อกซ์ ต่างที่เท็กซ์บ็อกซ์สามารถแสดงผลได้ทันทีแต่เท็กซ์ฟิลด์ต้องเอาไปใส่ไว้ในฟอร์มก่อนจึงจะแสดงผลได้

3.7 การใช้ API ระดับล่างในการพัฒนาส่วนติดต่อกับผู้ใช้

API ระดับล่างจะประกอบไปด้วยคลาส 2 ส่วนคือ คลาส แคนวาส(Canvas) ซึ่งมีชุดคำสั่งทางกราฟิกและเป็นคลาสที่สร้างอีเวนต์ระดับล่าง และคลาส กราฟิก(Graphic) ซึ่งใช้สร้างข้อความและรูปภาพหรือรูปต่าง ๆ

3.7.1 คลาสแคนวาส (cavas)

เป็นคลาสย่อยของ ดิสเพลย์เอเบิล(Displayable) คลาส ซึ่งใช้ API ระดับล่าง คลาสนี้มีเมธอดเพ้นท์(paint) ใช้สำหรับการวาด และเมธอดอื่น ๆ สำหรับจัดการอีเวนต์ระดับล่างซึ่งแอปพลิเคชันสามารถเขียนใหม่ได้ เมธอด Paint() ในคลาสแคนวาสถูกประกาศเป็น แอพสแต็ก(Abstract) เช่นเดียวกับคลาสแคนวาส คลาสใดที่ต้องการจัดการกับอีเวนต์ระดับล่างและเรียกใช้เมธอดทางกราฟิกเพื่อวาดบนจอจะต้องขยายนิยามของคลาส แคนวาสดังนี้

```
class TestCanvas extends Canvas{
```

```
...
```

```
void paint(Graphics g){
```

```
...
```

```

}
...
}

```

ขนาดของแคนवास

ขนาดที่แท้จริงของพื้นที่แสดงผลจะแตกต่างกันไปตามอุปกรณ์แต่ละชนิด ซึ่งแนะนำว่าโปรแกรมประยุกต์นั้น ๆ ไม่ควรกำหนดขนาดที่ตายตัวแต่ควรใช้เมธอด `getHeight()` และ `getWidth()`

เหตุการณ์ (Event)

จุดประสงค์หลักในการใช้คลาสแคนวาสก็เพื่อเข้าถึงและจัดการเหตุการณ์ระดับล่าง โปรแกรมประยุกต์สามารถเพิ่มคำสั่งต่าง ๆ ลงในแคนวาสได้โดยใช้เมธอด `addCommand()`

3.7.2 กราฟิกส์ (Graphics)

คลาสกราฟิกส์ไว้ทำหน้าที่สร้างภาพเรขาคณิต 2 มิติ คลาสนี้คล้ายกับ `java.awt.Graphics` ใน J2 สำหรับสร้างรูป ข้อความ ส่วนโค้งต่าง ๆ โดยมีตัวอย่างเช่น

```

void drawChar(char character,int x,int y,int anchor)
void drawChar(char[] data,int offset,int lenght,int x,int y,int anchor)
void drawImage(Image img,int x,int y,int anchor)
void drawLine(int x1,int y1, int x2,int y2)
void drawRect(int x,int y, int width,int height)
void drawRoundRect(int x,int y, int width,int height,int arcWidth,int arcHeight)
void drawString(String str,int x,int y,int anchor)
void fillArc(int x,int y,int width,int hight,int startAngle,int arcAngle)
void fillRect(int x,int y,int width,int hight)
void fillRoundRect(int x,int y, int width,int height,int arcWidth,int arcHeight)
int getColor()
Font getFont()
int getStrokeStyle()
int getTranslateX()
int getTranslateY()
void setClip(int x,int y,int width,int height)
void setColor(int RGB)
void setColor(int red,int green,int blue)

```

```
void translate(int x,int y)
```

โหมดการวาดของคลาส กราฟิกเท่านั้นที่เป็นแบบแทนที่พิกเซล พิกเซลที่มีอยู่เดิมบนจอจะถูกแทนที่ด้วยพิกเซลปัจจุบันที่ระบุในออบเจกต์กราฟิก(ไม่มีฟังก์ชันอื่นที่ทำหน้าที่ดึงรวมพิกเซล)

ระบบพิกัด

ระบบพิกัดของกราฟิกที่มีจุดปักเรียกว่าจุดสมอ(anchor) จะเริ่มที่มุมซ้ายของพื้นที่วาดค่า x จะเพิ่มจากซ้ายไปขวา และค่า y จะเพิ่มจากบนลงล่าง โดยระบบพิกัดนั้นจะอ้างอิงตำแหน่งระหว่างพิกเซล ไม่ใช่ตำแหน่งพิกเซลเอง

สี

API ระดับล่างใช้ระบบสีแบบ 24 บิต นั่นคือใช้ 8 บิตสำหรับแต่ละองค์ประกอบสี คือ สีแดง สีเขียว และสีน้ำเงิน เนื่องจากอุปกรณ์บางชนิดไม่สนับสนุนสีแบบ 24 บิต การนำไปใช้จึงมีการจับคู่สีที่โปรแกรมประยุกต์ต้องการใหม่ให้เป็นสีที่อุปกรณ์สามารถแสดงผลได้

ต่อไปนี้เป็นเมธอดสำหรับการเช็ดค่าและอ่านสี

```
int getColor()
```

```
int getBlueComponent()
```

```
int getGreenComponent()
```

```
int getRedComponent()
```

```
int getGrayScale()
```

```
void setColor(int RGB)
```

```
void setColor(int red,int green,int blue)
```

```
void setGrayScale(int value)
```

การวาดอักษร

คลาส กราฟิก มีเมธอดดังต่อไปนี้สำหรับวาดข้อความ

```
void drawChar(char character,int x,int y,int anchor)
```

```
void drawChars(char[] data,int offset,int length,int x,int y,int anchor)
```

```
void drawstring(String str,int x,int y,int anchor)
```

```
void drawSubstring(String str,int offset,int len,int x,int y,int anchor)
```

ฟอนต์

คลาส ฟอนต์ เป็นตัวแทนของฟอนต์และเมตริกซ์ของฟอนต์ โปรแกรมประยุกต์ใด ๆ ไม่สามารถสร้างฟอนต์ใหม่เองได้ ต้องใช้ฟอนต์ที่มีอยู่เดิม ซึ่งสามารถเรียกดูรายละเอียดต่าง ๆ ของฟอนต์ได้โดยใช้ เมธอดนี้

Static Font getFont(int face,int style,int size)

3.8 หน่วยเก็บข้อมูลแบบคงตัว

หน่วยเก็บข้อมูลแบบคงตัว ในที่นี้หมายถึงการเก็บข้อมูลลงในฐานข้อมูล ซึ่งตารางในฐานข้อมูลเราจะเรียกว่า เรคคอร์ดสตอร์(record store) แต่ละข้อมูลที่อยู่ใน เรคคอร์ดสตอร์ เราเรียกว่า เรคคอร์ด

เรคคอร์ดสตอร์ (Record store)

RMS ที่มีใน MIDP เป็นระบบฐานข้อมูลแบบง่าย ๆ เรคคอร์ดสตอร์ ซึ่งเทียบได้เท่ากับตารางฐานข้อมูลคือไฟล์ที่ประกอบด้วยชุดเรคคอร์ด

การทำงานของเรคคอร์ดสตอร์

คลาส RecordStore ได้จัดเตรียมวิธีการจัดการ เรคคอร์ดสตอร์ โดยมีคำสั่ง เปิด,ปิด และสร้าง ดังนี้

```
public static RecordStore openRecordStore(String recordStoreName,Boolean createIfNecessary)
```

```
public void closeRecordStore()
```

```
public static void deleteRecordStore(String recordStoreName)
```

MIDlet สร้าง เรคคอร์ดสตอร์ ขึ้นมาใหม่โดยใช้เมธอด

```
RecordStore.openRecordStore(true)
```

แต่ถ้า recordStoreName ไม่มีอยู่ก่อน ก็จะสร้างมาใหม่ และหากมีอยู่แล้วก็จะเป็นการเปิดขึ้นมา

การจัดการ เรคคอร์ด

กลุ่มของข้อมูล เรคคอร์ดสตอร์ เราเรียกว่า เรคคอร์ดซึ่ง โครงสร้างของเรคคอร์ดเกิดจากอาร์เรย์ของไบต์ที่จะกำหนดค่าสำหรับแต่ละเรคคอร์ดด้วย RecordTD เรคคอร์ดสามารถสร้างใหม่ ลบ เรียกดู หรือแก้ไขได้ก็ต่อเมื่อได้เปิด เรคคอร์ดสตอร์ ด้วยเมธอดดังนี้

```

public int addRecord(byte[] data,int offset,int numBytes)

public void deleteRecord(int recordid)

public int getRecord(int recorded,byte[] buffer,int offset)

public byte[] getRecord(int recordId)

public void setRecord(int recorded,byte[] newData,int offset,int numByte)

```

3.9 การทำงานของส่วนการเชื่อมต่อเครือข่าย

ในส่วนนี้เราจะกล่าวถึงการติดต่อเครือข่ายสำหรับอุปกรณ์ไร้สายโดยใช้ HttpURLConnection ซึ่งจุดประสงค์ในการนำ HttpURLConnection มาใช้งานในการติดต่อสื่อสารในแอปพลิเคชันมีข้อดีหลายประการด้วยกันเช่น

- อุปกรณ์ MIDP บางตัวไม่สนับสนุนการติดต่อสื่อสารแบบซ็อกเก็ตและดาต้าแกรม ขณะที่อุปกรณ์ทุกตัวรองรับการติดต่อแบบ HTTP อยู่แล้ว
- ในโครงงานนี้มีความจำเป็นที่ต้องร้องขอเครื่องให้บริการ(server) เพื่ออ่านเอกสาร XML ดังนั้นการใช้ HTTP จึงสะดวกที่สุด

ในการสร้าง การร้องขอผ่าน HTTP นั้นจะใช้ เมธอด Connector.open ซึ่งเราสามารถเขียนได้ดังนี้

```
import javax.microedition.io.*;
```

```
HttpConnection conn = Connector.open( "http://www.java.sun.com/jdc" );
```

แต่หากมีการ ส่งต่อ(redirect) เกิดขึ้น วิธีนี้ก็ไม่สามารถไปยัง URL ที่ให้ส่งต่อไปได้ ซึ่งเราต้องเขียน Class ขึ้นมาเองซึ่งจะไม่ขออธิบายไว้ในที่นี้